

Lekcja

Temat: Instrukcja DQL (ang. Data Querying Language) - SELECT, order by, LIMIT, OFFSET, WHERE, GROUP BY

```
DROP TABLE IF EXISTS pracownicy;
```

```
-- Tworzenie tabeli
```

```
CREATE TABLE pracownicy (  
    imie          VARCHAR(50) NOT NULL,  
    nazwisko      VARCHAR(50) NOT NULL,  
    departament  VARCHAR(50),  
    pensja       DECIMAL(10,2)  
);
```

```
-- Wstawianie danych
```

```
INSERT INTO pracownicy (imie, nazwisko, departament, pensja) VALUES  
( 'Jan',      'Kowalski',      'IT',      5000.00),  
( 'Anna',     'Nowak',         'HR',      4500.00),  
( 'Piotr',    'Zieliński',    'IT',      6200.00),  
( 'Maria',    'Wiśniewska',   'Finanse', 7000.00),  
( 'Tomasz',   'Lewandowski',  'IT',      5500.00),  
( 'Katarzyna', 'Kamińska',    'HR',      5200.00),  
( 'Adam',     'Nowicki',     'Finanse', 4800.00),  
( 'Ola',      'Dąbrowska',   'IT',      6500.00),  
( 'Robert',   'Malinowski',  'IT',      NULL);
```

ORDER BY

Instrukcja ORDER BY w zapytaniu SELECT służy do **sortowania** wyników zapytania. Bez niej wiersze są zwracane w **nieokreślonej kolejności** (zależnej od silnika bazy danych i fizycznego układu danych na dysku). ORDER BY jest **ostatnią** klauzulą w zapytaniu (przed LIMIT/OFFSET).

Składnia:

```
SELECT kolumny  
FROM tabela
```

```
[WHERE warunek]
ORDER BY kolumna1 [ASC|DESC], kolumna2 [ASC|DESC], ...
```

- **ASC** – rosnąco (domyślne, można pominąć)
- **DESC** – malejąco

Przykłady:

```
-- Pracownicy posortowani alfabetycznie po nazwisku (rosnąco)
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko;
```

```
-- To samo, ale malejąco
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko DESC;
```

```
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY nowa_pensja DESC;          -- można użyć aliasu
```

```
-- lub po pozycji kolumny w SELECT (niezalecane, ale działa)
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY 3 DESC;                   -- sortuje po 3. kolumnie (pensja)
```

Sortowanie z NULL

Różne bazy danych traktują *NULL* inaczej:

PostgreSQL, SQL Server, SQLite, MySQL, MariaDB - *NULL* jako najmniejsza wartość (na początku przy ASC)

Inne sposoby sortowania:

ORDER BY RANDOM() - losowa kolejność (PostgreSQL)

ORDER BY 1, 2 - sortowanie po pierwszej i drugiej kolumnie

LIMIT i OFFSET

LIMIT 3 OFFSET 4 → weź 3, pomiń 4

LIMIT 3, 4 → pomiń 3, weź 4

- Sposób 1 (z OFFSET – bardziej czytelny)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 3 OFFSET 4;
```

-- Sposób 2 (z przecinkiem – najpopularniejszy)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 4, 3;      -- UWAGA: najpierw offset, potem count!
```

WHERE

WHERE w MySQL to **filtr wierszy** – działa **przed** grupowaniem (GROUP BY) i mówi bazie: „pokaż tylko te rekordy, które spełniają warunek”.

Składnia:

```
SELECT kolumny
FROM tabela
WHERE warunek      -- ← tutaj filtrujemy
[GROUP BY ...]
[HAVING ...]
ORDER BY ...;
```

Kolejność wykonywania zapytania:

1. FROM (skąd dane)
2. WHERE ← filtrujemy wiersze
3. GROUP BY
4. HAVING
5. SELECT, ORDER BY, LIMIT

Najważniejsze operatory w WHERE

Operator	Znaczenie	Przykład
=	równa się	departament = 'IT'
!= lub <>	różne od	pensja != 5000
> / < / >= / <=	porównanie liczb	pensja > 6000
LIKE	wyszukiwanie tekstu (z % i _)	nazwisko LIKE 'K%'
IN	w liście wartości	departament IN ('IT', 'HR')
BETWEEN	w zakresie	pensja BETWEEN 5000 AND 6000
IS NULL / IS NOT NULL	NULL-e	pensja IS NULL
AND / OR / NOT	łączenie warunków	pensja > 5000 AND departament = 'IT'

Przykłady:

1. Prosty filtr – tylko jeden dział

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT';
```

2. Więcej niż jedna warunek (AND)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT'
AND pensja > 6000;
```

3. Albo jeden, albo drugi (OR)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament = 'HR'
OR departament = 'Finanse';
```

4. Wyszukiwanie tekstowe (LIKE)

```
SELECT imie, nazwisko
FROM pracownicy
WHERE nazwisko LIKE 'K%';
-- albo
WHERE nazwisko LIKE '%ski';
```

5. Zakres pensji

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE pensja BETWEEN 5000 AND 6000;
```

6. Rekordy z NULL

```
SELECT imie, nazwisko
FROM pracownicy
WHERE pensja IS NULL;
```

7. Kilka działów naraz (IN)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament IN ('IT', 'HR');
```

Różnica WHERE vs HAVING

- WHERE – filtruje **pojedyncze wiersze** (przed grupowaniem)
- HAVING – filtruje **całe grupy** (po GROUP BY)

```
-- WHERE filtruje przed grupowaniem
SELECT departament, COUNT(*)
FROM pracownicy
WHERE pensja > 5000          -- tylko osoby z pensją > 5000
GROUP BY departament;

-- HAVING filtruje po grupowaniu
SELECT departament, AVG(pensja)
FROM pracownicy
GROUP BY departament
HAVING AVG(pensja) > 5000;   -- tylko działy ze średnią > 5000
```

GROUP BY

GROUP BY w **MySQL** służy do **grupowania wierszy**, które mają te same wartości w wybranych kolumnach, a potem wykonywania na tych grupach operacji agregujących (np. liczenie, sumowanie, średnia).
Bez GROUP BY agregaty (COUNT, SUM, AVG...) **liczą wszystko razem**.
Z GROUP BY – liczą **osobno dla każdej grupy**.

Przykłady.

1. Ile osób pracuje w każdym dziale? (najpopularniejsze użycie)

```
SELECT
    departament,
    COUNT(*) AS liczba_pracownikow
FROM pracownicy
GROUP BY departament;
```

2. Średnia pensja w każdym dziale

```
SELECT
    departament,
    ROUND(AVG(pensja), 2) AS srednia_pensja,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament;
```

3. Suma wszystkich pensji w każdym dziale

```
SELECT
    departament,
    SUM(pensja) AS suma_pensji,
    MIN(pensja) AS najnizsza,
    MAX(pensja) AS najwyzsza
FROM pracownicy
GROUP BY departament;
```

4. Tylko działy, w których pracuje więcej niż 3 osoby (HAVING)

```
SELECT
    departament,
    COUNT(*) AS liczba
FROM pracownicy
GROUP BY departament
HAVING liczba > 3;          -- filtrujemy grupy!
```

5. Filtrujemy przed grupowaniem (WHERE)

```
SELECT
    departament,
    COUNT(*) AS ile,
    AVG(pensja) AS srednia
FROM pracownicy
WHERE pensja IS NOT NULL      -- pomijamy rekord z NULL
GROUP BY departament
HAVING AVG(pensja) > 5000;
```

6. Grupowanie po więcej niż jednej kolumnie

```
SELECT
    departament,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament, imie;
```

Lekcja

Temat: Having

Having

Having to **klauzura** w bazach danych (nie samodzielne polecenie ani instrukcja). Jest **częścią zapytania SELECT**. Działa **tylko razem z GROUP BY**. Służy do **filtrowania całych grup** po tym, jak dane zostały zgrupowane i policzone funkcje agregujące (COUNT SUM, AVG, MAX, MIN itp.)

Składnia:

```
SELECT kolumna1, funkcja_agregująca(kolumna2)
FROM tabela
[WHERE warunek_na_wiersze]
GROUP BY kolumna1
HAVING warunek_na_grupy;
```

Porównanie HAVING z WHERE

- WHERE filtruje **pojedyncze wiersze przed** grupowaniem.
- HAVING filtruje **całe grupy po** grupowaniu.

```
CREATE TABLE IF NOT EXISTS employees (
    id            INT PRIMARY KEY,
    first_name    VARCHAR(50),
    department    VARCHAR(50),
    salary        DECIMAL(10,2)
);
```

```
INSERT INTO employees (id, first_name, department, salary) VALUES
(1, 'Jan', 'IT', 12000.00),
(2, 'Anna', 'IT', 15000.00),
(3, 'Piotr', 'IT', 8000.00),
(4, 'Maria', 'HR', 9000.00),
(5, 'Tomasz', 'HR', 11000.00),
(6, 'Kasia', 'HR', 9500.00),
(7, 'Michał', 'Sales', 20000.00),
(8, 'Ola', 'Sales', 18000.00),
(9, 'Adam', 'Marketing', 7000.00),
(10, 'Ewa', 'Marketing', 8500.00),
(11, 'Kamil', 'IT', 14000.00),
(12, 'Natalia', 'IT', 13000.00),
```

```
(13, 'Robert', 'HR', 10500.00),
(14, 'Sylwia', 'Sales', 16000.00),
(15, 'Łukasz', 'Marketing', 7500.00),
(16, 'Paweł', 'IT', 9500.00),
(17, 'Zofia', 'IT', 12500.00);
```

```
CREATE TABLE IF NOT EXISTS orders (
    order_id      INT PRIMARY KEY,
    customer_id   INT,
    product_name  VARCHAR(100),
    price         DECIMAL(10,2),
    quantity      INT,
    total_amount  DECIMAL(10,2)
);
```

```
INSERT INTO orders (order_id, customer_id, product_name, price, quantity,
total_amount) VALUES
(1, 101, 'Laptop', 4500.00, 1, 4500.00),
(2, 101, 'Smartphone', 2500.00, 1, 2500.00),
(3, 101, 'Headphones', 300.00, 2, 600.00),
(4, 101, 'Laptop', 4200.00, 1, 4200.00),
(5, 101, 'Monitor', 1200.00, 1, 1200.00),
(6, 102, 'Laptop', 4300.00, 1, 4300.00),
(7, 102, 'Smartphone', 2800.00, 1, 2800.00),
(8, 102, 'Tablet', 1800.00, 1, 1800.00),
(9, 102, 'Laptop', 4600.00, 1, 4600.00),
(10, 103, 'Headphones', 250.00, 3, 750.00),
(11, 103, 'Smartphone', 2200.00, 1, 2200.00),
(12, 104, 'Monitor', 1100.00, 1, 1100.00),
(13, 101, 'Tablet', 1700.00, 1, 1700.00),
(14, 105, 'Laptop', 5000.00, 1, 5000.00),
(15, 102, 'Headphones', 350.00, 2, 700.00),
(16, 101, 'Monitor', 1300.00, 1, 1300.00),
(17, 106, 'Smartphone', 2400.00, 1, 2400.00),
(18, 101, 'Laptop', 4800.00, 1, 4800.00),
(19, 102, 'Tablet', 1900.00, 1, 1900.00),
(20, 103, 'Monitor', 1150.00, 1, 1150.00);
```

Przykłady

1: Liczba pracowników w departamentach (tylko te z > 5 osobami)

```
SELECT
    department,
    COUNT(*) AS liczba_pracownikow
```



```
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

2: Produkty, które sprzedały się za więcej niż 10 000 zł

```
SELECT
    product_name,
    SUM(price * quantity) AS suma_sprzedazy
FROM orders
GROUP BY product_name
HAVING SUM(price * quantity) > 10000;
```

3: Klienci, którzy złożyli więcej niż 3 zamówienia (i średnia wartość zamówienia > 500)

```
SELECT
    customer_id,
    COUNT(order_id) AS liczba_zamowien,
    AVG(total_amount) AS srednia_wartosc
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 3
    AND AVG(total_amount) > 500;
```

4: Działy, w których maksymalna pensja przekracza 15 000 zł

```
SELECT
    department,
    MAX(salary) AS max_pensja
FROM employees
GROUP BY department
HAVING MAX(salary) > 15000;
```

5: Departamenty z więcej niż 5 pracownikami

```
SELECT
    department,
    COUNT(*) AS liczba_pracownikow
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

6: Produkty, które sprzedały się za więcej niż 10 000 zł

```
SELECT
    product_name,
    SUM(price * quantity) AS suma_sprzedazy
FROM orders
GROUP BY product_name
HAVING SUM(price * quantity) > 10000;
```

7: Klienci z > 3 zamówieniami i średnią wartością zamówienia > 500 zł

```
SELECT
    customer_id,
    COUNT(order_id) AS liczba_zamowien,
    AVG(total_amount) AS srednia_wartosc
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 3
    AND AVG(total_amount) > 500;
```

8: Działy, w których maksymalna pensja przekracza 15 000 zł

```
SELECT
    department,
    MAX(salary) AS max_pensja
FROM employees
GROUP BY department
HAVING MAX(salary) > 15000;
```

Lekcja

Temat: Sprawdzian wiadomości

Lekcja - JUTRO oddanie projektów

Temat: JOIN w SQL

```
CREATE TABLE klienci (
    klient_id INT PRIMARY KEY,
    imie      VARCHAR(50),
    miasto    VARCHAR(50)
);
```

```
INSERT INTO klienci VALUES
(1, 'Jan Kowalski', 'Warszawa'),
(2, 'Anna Nowak', 'Kraków'),
(3, 'Piotr Wiśniewski', 'Gdańsk'),
(4, 'Maria Zielińska', 'Wrocław'),
(5, 'Tomasz Lewandowski', 'Poznań');
```

```
CREATE TABLE zamowienia (  
    zamowienie_id INT PRIMARY KEY,  
    klient_id INT,  
    kwota DECIMAL(10,2),  
    data_zamowienia DATE  
);
```

```
INSERT INTO zamowienia VALUES  
(101, 1, 450.00, '2025-01-15'),  
(102, 2, 1200.50, '2025-02-10'),  
(103, 1, 320.00, '2025-02-20'),  
(104, 3, 890.00, '2025-03-05'),  
(105, 6, 150.00, '2025-03-10'); -- klient_id = 6 → nie istnieje!
```

INNER JOIN

Zwraca **tylko wiersze, które mają dopasowanie w obu tabelach.**

```
SELECT  
    k.imie,  
    k.miasto,  
    z.zamowienie_id,  
    z.kwota,  
    z.data_zamowienia  
FROM klienci k  
INNER JOIN zamowienia z ON k.klient_id = z.klient_id;
```

imie	miasto	zamowienie_id	kwota	data_zamowienia
Jan Kowalski	Warszawa	101	450.00	2025-01-15
Anna Nowak	Kraków	102	1200.50	2025-02-10
Jan Kowalski	Warszawa	103	320.00	2025-02-20
Piotr Wiśniewski	Gdańsk	104	890.00	2025-03-05

LEFT JOIN (LEFT OUTER JOIN)

Zwraca **wszystkich klientów** + ich zamówienia (jeśli nie ma zamówienia → NULL).

```
SELECT
  k.imie,
  k.miasto,
  z.zamowienie_id,
  z.kwota
FROM klienci k
LEFT JOIN zamowienia z ON k.klient_id = z.klient_id
ORDER BY k.klient_id;
```

imie	miasto	zamowienie_id	kwota
Jan Kowalski	Warszawa	101	450.00
Jan Kowalski	Warszawa	103	320.00
Anna Nowak	Kraków	102	1200.50
Piotr Wiśniewski	Gdańsk	104	890.00
Maria Zielińska	Wrocław	NULL	NULL
Tomasz Lewandowski	Poznań	NULL	NULL

RIGHT JOIN (RIGHT OUTER JOIN)

Zwraca **wszystkie zamówienia** + dane klienta (jeśli klienta nie ma → NULL).

```
SELECT
    k.imie,
    k.miasto,
    z.zamowienie_id,
    z.kwota
FROM klienci k
RIGHT JOIN zamowienia z ON k.klient_id = z.klient_id
ORDER BY z.zamowienie_id;
```

imie	miasto	zamowienie_id	kwota
Jan Kowalski	Warszawa	101	450.00
Anna Nowak	Kraków	102	1200.50
Jan Kowalski	Warszawa	103	320.00
Piotr Wiśniewski	Gdańsk	104	890.00
NULL	NULL	105	150.00

Złota zasada

- **Lewa tabela (left table)** = tabela, która stoi **bezpośrednio po słowie FROM**
- **Prawa tabela (right table)** = tabela, która stoi **bezpośrednio po słowie JOIN** (lub LEFT JOIN, RIGHT JOIN, INNER JOIN itp.)

Lekcja - oddanie projektów

Temat: Weryfikacja indywidualnych projektów

